数据库项目面试 Q&A

> 基于 HDIS 血液透析系统、药房模块等实战项目整理

一、性能优化案例

Q1：你把 HDIS 报表查询从 1000ms 优化到 200ms，用了什么方法？

参考回答：

主要从三个层面优化：

1. **索引优化**
   - 分析慢 SQL 执行计划，定位全表扫描
   - 针对高频筛选条件建立复合索引
   - 使用覆盖索引避免回表

```sql
-- 优化前：全表扫描
SELECT * FROM DialysisReport
WHERE DepartmentId = @deptId AND ReportDate BETWEEN @start AND @end
ORDER BY ReportDate DESC

-- 优化后：复合索引 + 覆盖索引
CREATE INDEX IX_DialysisReport_Dept_Date
ON DialysisReport(DepartmentId, ReportDate DESC)
INCLUDE (PatientId, TreatmentType, Amount);
```

1. **缓存策略**
   - Redis 缓存热点报表数据
   - 设置合理 TTL（如 5 分钟）
   - 缓存 key 按维度组合避免穿透

```csharp
var cacheKey = $"report:{deptId}:{date:yyyyMMdd}";
var cached = await redis.StringGetAsync(cacheKey);
if (!cached.IsNullOrEmpty) return JsonSerializer.Deserialize<Report>(cached);
```

1. **查询优化**
   - 避免 SELECT *，只查询必要字段
   - 复杂统计用物化视图或定时任务预计算

效果：响应时间从约 1000ms 降至约 200ms

Q2：如何判断一个 SQL 需不需要加索引？

参考回答：

1. **看执行计划**：关注 `Scan`（全表扫描）、`Sort`、`Key Lookup`（回表）
2. **查慢查询日志**：统计高频慢 SQL
3. **分析查询条件**：WHERE/JOIN/ORDER BY 使用的字段
4. **权衡读写**：索引过多影响写入性能

```sql
-- SQL Server 查看执行计划
SET STATISTICS IO ON;
SET STATISTICS TIME ON;
SELECT * FROM Orders WHERE UserId = 1;
```

Q3：复合索引的最左前缀原则是什么？

参考回答：

复合索引 `(A, B, C)` 相当于创建了 `(A)`、`(A, B)`、`(A, B, C)` 三个索引。

查询条件必须包含最左边的列，才能使用索引：

```sql
-- 能使用索引
WHERE A = 1
WHERE A = 1 AND B = 2
WHERE A = 1 AND B = 2 AND C = 3

-- 无法使用索引（跳过 A）
WHERE B = 2
WHERE B = 2 AND C = 3
```

二、缓存应用

Q4：MongoDB 做第三方接口缓存，TTL 怎么设计？

参考回答：

1. 根据接口数据更新频率设置 TTL
2. 相同请求优先查缓存，减少第三方调用
3. TTL 过期后自动删除，无需手动清理

```mongo
// MongoDB TTL 索引
db.thirdPartyCache.createIndex(
    { "expireAt": 1 },
    { expireAfterSeconds: 0 } // 到达 expireAt 后立即删除
)
```

```dotnet
// C# 写入缓存
await collection.InsertOneAsync(new ThirdPartyCache
{
    Key = requestKey,
    Value = response,
    ExpireAt = DateTime.UtcNow.AddMinutes(5)
});
```

效果：第三方 API 调用次数下降约 50%

Q5：Redis 缓存和本地缓存如何选择？

特性	Redis 分布式缓存	本地内存缓存
共享性	多节点共享	仅当前进程
容量	受限于 Redis 内存	受限于应用内存
速度	网络 I/O（更快）	无网络开销（最快）
适用场景	多实例部署、共享数据	进程内高频访问的字典数据

项目中实践：

- Redis：报表结果、热点业务数据
- 本地缓存：科室字典、药品基础信息（变化少、访问极高）

三、事务与并发

Q6：高并发下库存扣减如何保证不超卖？

方案一：数据库乐观锁

```
// UPDATE 返回受影响行数判断
var rows = await db.ExecuteAsync(@"
    UPDATE Products
    SET Stock = Stock - 1, Version = Version + 1
    WHERE Id = @id AND Version = @version AND Stock > 0",
    new { id, version });

if (rows == 0) throw new Exception("库存不足或并发冲突");
```

方案二：Redis 原子操作

```
var result = await redis.ExecuteAsync("DECR", $"inventory:{productId}");
if (result < 0)
{
    // 库存不足，回滚
    await redis.IncrAsync($"inventory:{productId}");
    throw new Exception("库存不足");
}
```

项目中实践：先扣 Redis 库存，异步同步到数据库，数据库失败则回滚 Redis

Q7：什么是脏读、不可重复读、幻读？

现象	解释	隔离级别
脏读	读取到其他事务未提交的数据	读未提交
不可重复读	同一行数据两次读取结果不同	读已提交
幻读	同条件两次查询行数增加/减少	可重复读

```
-- 幻读演示
SELECT COUNT(*) FROM Orders WHERE Status = 'Pending'; -- 第一次：10 条
-- 另一个事务插入一条 Pending 订单
SELECT COUNT(*) FROM Orders WHERE Status = 'Pending'; -- 第二次：11 条
```

四、多租户与数据隔离

Q8：多租户数据隔离有哪几种方案？

方案	实现方式	适用场景
字段隔离	所有表添加 TenantId	数据量中等、查询频率高
Schema 隔离	每个租户独立 Schema	数据量大、需要较强隔离
数据库隔离	独立数据库	数据完全独立、隔离性要求高

项目中实践：采用字段隔离，在框架层统一注入租户过滤条件，业务代码无感知

```
// 租户过滤
public class TenantFilter : IQueryFilter
{
    public Expression<Func<T, bool>> Build<T>()
    {
        return e => e is ITenantEntity tenant && tenant.TenantId == CurrentTenant.Id;
    }
}
```

Q9：报表数据量很大，时间久了怎么归档？

参考回答：

1. **分区表**：按月份/年份分区，查询只扫相关分区
2. **冷热分离**：历史数据迁移到归档库/对象存储
3. **物化视图**：预计算聚合结果，报表查询直接查视图

```
-- 按月分区（SQL Server）
CREATE PARTITION FUNCTION pf_OrderDate (DATE)
AS RANGE RIGHT FOR VALUES
('2024-01-01', '2024-02-01', '2024-03-01', '2024-04-01');
```

```
-- 归档示例
INSERT INTO Orders_Archive SELECT * FROM Orders WHERE CreatedAt < '2023-01-01';
DELETE FROM Orders WHERE CreatedAt < '2023-01-01';
```

五、设计与规范化

Q10：药品批次管理如何设计？

需求：效期跟踪、FIFO 出库、批次追溯

表结构设计：

```
-- 药品批次表
CREATE TABLE ProductBatches (
    Id BIGINT PRIMARY KEY,
    ProductId BIGINT NOT NULL,            -- 药品ID
```

```sql
    BatchNo NVARCHAR(50) NOT NULL,        -- 批号
    ProductionDate DATE NOT NULL,         -- 生产日期
    ExpiryDate DATE NOT NULL,             -- 有效期至
    Quantity DECIMAL(18,2) NOT NULL,      -- 当前库存
    UnitPrice DECIMAL(18,4) NOT NULL,     -- 进货单价
    WarehouseId BIGINT NOT NULL,          -- 仓库
    TenantId BIGINT NOT NULL,
    CreatedAt DATETIME2 NOT NULL DEFAULT GETDATE(),
    INDEX IX_Product_Expiry (ProductId, ExpiryDate) -- FIFO 查询
);
```

出库逻辑：优先消耗有效期最近的批次

```csharp
var batch = await db.Queryable<ProductBatch>()
    .Where(x => x.ProductId == productId && x.Quantity > 0)
    .OrderBy(x => x.ExpiryDate) // FIFO：按效期排序
    .FirstAsync();
```

Q11：处方流转数据如何保证一致性？

场景：医生站开处方 → 收费 → 药房发药

方案：

1. **事务**：同一数据库内使用事务
2. **消息队列**：跨系统通过 MQ 保证最终一致性
3. **状态机**：处方状态流转严格控制

```csharp
// 处方状态
public enum PrescriptionStatus
{
    Created,     // 已创建
    Paid,        // 已缴费
    Dispensing,  // 配药中
    Dispensed,   // 已发药
    Cancelled    // 已作废
}

// 状态流转校验
public bool CanTransition(PrescriptionStatus from, PrescriptionStatus to)
{
    return (from, to) switch
    {
        (PrescriptionStatus.Created, PrescriptionStatus.Paid) => true,
        (PrescriptionStatus.Paid, PrescriptionStatus.Dispensing) => true,
        (PrescriptionStatus.Dispensing, PrescriptionStatus.Dispensed) => true,
        _ => false
    };
}
```

六、反问环节

Q12：数据库岗位可能问的问题

1. 你们数据量多大？有没有分库分表？
2. 主从复制怎么做？同步延迟怎么处理？
3. 慢查询如何定位和优化？
4. 数据备份策略是什么？
5. 有没有做读写分离？

建议：结合 HDIS 项目，重点准备索引优化、缓存策略、多租户隔离三个方向，能说出具体问题→分析→解决的完整思路